GymTS接口规范

GymTS 接收实验计划，通过 EnvironmentTS 运行实验并保存证据，交付 RunResult；BenchTS 使用固定评价方案和独立结果依据生成 EvaluationReport。 本文集中定义这条流程的服务入口、配置、数据对象和内部调用契约，其他文档以此为准。签名表达逻辑接口，不表示已有 SDK 实现。

GymTS 的主要模块只有 EnvironmentTS 和 BenchTS。Runner、Environment、Task、ExperimentRecords 是 EnvironmentTS 内部执行与记录实验所用的对象；Candidate 是运行时通过适配调用的算法、模型、流水线或 Agent。基础调用为进程内同步、单实例串行；底层服务可通过平台或本地适配实现。

1. 对外服务：提交计划，取得结果与证据

接口	调用方	作用与返回
GymTS.run(spec: ExperimentSpec) -> RunResult	LabTS、上层流程或实验脚本	公共运行入口；将计划交给 EnvironmentTS，返回任务结果、制品及记录入口。
EnvironmentTS.run(spec: ExperimentSpec) -> RunResult	GymTS 门面；BenchTS 内部案例执行	执行一次实验。它是模块内部的执行入口，不是另一套运行机制。
BenchTS.evaluate(spec: EvaluationSpec) -> EvaluationReport	LabTS 的比较、验收、回归流程	按固定案例执行候选和基线，评分并交付报告。
BenchTS.score(spec: ScoreSpec) -> EvaluationReport	使用已有运行结果的调用方	读取保存的原始结果重新评分，不重新调用被测对象。
EnvironmentTS.records.get_run(run_id : str) -> RunRecord	LabTS/EvolveTS、BenchTS、获准的研发工具	读取一次 Run 的元数据、结果摘要和事件入口。
EnvironmentTS.records.read_events(run_id: str, query: EventQuery) -> EventPage	同上	按类型、任务或时间读取实验事件和制品引用。

一次 Run 有明确范围或结束条件。批量、流式、交互运行都使用同一个入口。完成 Run、达到评价门槛和获得交付批准是三个不同结果：前两者由 GymTS 交付，版本选择、评审与批准由 LabTS 完成。

2. 实验计划、数据视图与制品引用

2.1 ExperimentSpec 的组成

代码块

```
ExperimentSpec
  name
  scene: {environment: EnvironmentSpec, task: TaskSpec, binding: BindingSpec}
```

```
    candidate: CandidateSpec
    execution: ExecutionSpec
    recording: RecordingSpec
```

scene 可内联或引用已登记的场景版本，进入运行前解析为完整配置。name 是可选的可读名称。adapter/plugin 与 version 固定所选扩展；凭证通过现场服务解析，不写入普通配置快照。

对象	字段与含义
EnvironmentSpec	protocol_version、adapter、version、source: SourceSpec；mapping 为原始字段/实体/时间/操作映射，options 为静态连接与读取选项，后两项缺省 {}。
SourceSpec	kind: dataset/simulator/gym/live、locator 必需；revision、credential_ref 缺省 null，connection 缺省 {}，只含非秘密参数。数据集固定版本；实时源记录实际系统标识和接入时间。
TaskSpec	plugin、version、params；可附任务公开的说明/上下文引用。version 是实现版本，任务数据协议版本另由 TaskDescription 声明。
BindingSpec	variables 将任务变量对应到环境变量，并说明单位、时间含义和必要变换；operations 映射任务操作及参数，缺省 {}。需要重采样时明确聚合/插值规则。
CandidateSpec	adapter、version、artifact: ArtifactRef或null、params（缺省 {}）；无需额外制品的函数或固定服务可使用 null。
RecordingSpec	inputs、outputs、artifacts 说明完整保存或保存引用；可要求被测对象内部阶段记录。使用 reference 时必须能恢复本次实际输入/输出。

同一原始字段的解析归 Environment 适配器；环境变量到公共任务变量的单位换算、窗口和重采样归 Task/Binding；方法自身的插补、特征与模型处理保留在其适配内部。采样周期本身不隐含平均、求和或插值。

2.2 ExecutionSpec：本次怎么运行

字段	取值与规则
mode	batch / stream / interactive：整段任务、逐批任务、可继续查询或操作的任务。
start	{mode: attach/reinitialize, seed: null或值, init: object}；seed/init 属于本次初始化，不放入 source/options。
scope	{start,end} 或 {duration: Duration}；前者为半开区间，后者在启动时由实际起点换算。控制环境的最终边界状态按第 3 节保留。
learning	frozen / fit_once / online：固定方法；准备时拟合一次；按允许反馈在线更新。
training_view	DataViewRef或null；fit_once 必需，online 是否准备训练由计划声明。
training_targets	{artifact: ArtifactRef, alignment_keys: list[string]}或null；需要监督训练时与 training_view 一起指定。标签制品的 schema_ref 定义字段、类型及目标含义。
feedback_until	可选截止时点或延长时长；停止产生新结果/动作后，可继续收集结果到何时。缺省不延长 scope。
default_command	可选 {operation,target,arguments}；纯观察任务使用 on_action 后端时，显式指定推进动作。
resource_policy_ref	可选引用；由 Agent Platform/现场设施落实已有资源设置，不自行授予权限。

Runner 将 training_view 交给 prepare，将 training_targets 放入准备上下文。适配器按 alignment_keys 将训练输入与标签配对，例如实体、变量、目标时点。无监督训练无需标签；运行后的在线反馈另走 CandidateFeedback，BenchTS 私有评价依据不自动用作训练标签。

2.3 引用与现场服务

对象	固定字段与含义
ArtifactRef	uri、revision、digest、format、schema_ref；uri/format 必需，revision/digest 可为 null 但至少一个提供不可变身份。schema_ref 为固定结构版本，非结构化制品可为 null。
DataViewRef	uri、revision、format、selection: {variables,entities,range}、visibility_cutoff、schema_ref；固定且已裁剪的数据视图，只能读取选择范围内的数据。
Duration	{value,unit}；value>0，unit 为 s 或 step；step 不未经声明换算成秒。

配置可使用逻辑引用字符串，装配时先解析固定版本再调用和记录。一个可变文件路径不等于不可变制品；DataViewRef 也不等于开放整个数据仓库。所有 schema_ref 指向具体版本的结构说明，明确字段类型和必需性。

`EnvironmentServices` 由 EnvironmentTS 注入，适配现场的 `resolve(ref) -> LocalResource`、`credential(ref) -> Credential`、`read_view(ref: DataViewRef) -> TimeSeriesData`、`save_artifact(content, metadata) -> ArtifactRef`。Environment 适配器使用连接/凭证能力；Task 经 TaskExecutionContext.data 仅获得受限 read_view；被测对象仅获得本次制品和获准视图的读取、保存能力。接口需对接实际平台或本地服务，不假定某平台已原生实现同名方法。

3. 环境接入：把后端变成可运行、可查询的实验环境

3.1 登记、装配与五个方法

代码块

```
class EnvironmentRegistry:
    def register(self, adapter: str, version: str,
                 factory: EnvironmentFactory) -> None: ...
    def create(self, spec: EnvironmentSpec,
               services: EnvironmentServices) -> Environment: ...

class EnvironmentFactory:
    def create(self, spec: EnvironmentSpec,
               services: EnvironmentServices) -> Environment: ...

class Environment:
    def describe(self) -> EnvironmentDescription: ...
    def start(self, spec: EnvironmentStart) -> SessionInfo: ...
    def observe(self, request: ObservationRequest) -> ObservationBatch: ...
    def act(self, command: EnvironmentCommand) -> ActionReceipt: ...
    def finish(self) -> EnvironmentSummary: ...
```

方法	正常调用时机与效果
register / create	扩展包装载时登记固定 adapter/version 与工厂；同版本不覆盖。Runner 每 Run 创建一个未启动实例，create 不重置或推进后端。
describe	start 前读取，可重复；公布变量、操作、时钟与有效能力，不开始实验。
start	绑定成功后调用一次，初始化或接入会话；首批数据由 observe 交付。
observe	Runner 按 Task 的读取请求调用，取得数据、反馈、进展或结束信息。
act	Runner 执行 Task 映射后的操作，返回即时回执，后续反馈通过 observe 交付。
finish	收尾时释放拥有的实例或脱离已接入系统，返回摘要。

每个 Environment 实例只拥有一个会话。装配顺序为：

> 解析 source/mapping/options → create → describe → Task/Binding 检查变量语义和可用能力 → 编译 EnvironmentStart → start → 保存配置与 SessionInfo。只读回放的 operations=[]，名称相同不意味着单位、时间尺度和控制能力相同。

3.2 describe：环境提供什么

EnvironmentDescription 字段	内容
protocol_version、adapter、version	协议版本与适配实现身份。
environment_id、source_revision	后端稳定标识；实际数据/系统版本，未知为 null。
variables	list[VariableSpec]；公开观测变量，不混入私有评价真值。
entities	list[EntitySpec] 或 EntityCatalogRef。
operations	list[OperationSpec]；只读环境为 []。
clock、capabilities	ClockSpec 与 Capabilities，见下表。
config_schema_ref	本适配版本的 source/mapping/options/init 结构。
context_schema_ref、feedback_schema_ref	公共上下文和扩展反馈结构，无相应内容时为 null。

描述对象	字段与规则
VariableSpec	name,dtype,shape,unit,description；shape 表示单时点值的形状，标量为 []；未知单位为 unknown，无量纲为 1。可附采样周期说明，不承诺规则采样。
EntitySpec / EntityCatalogRef	实体为 id,type,description；目录引用为 uri,revision，解析成实体列表。
OperationSpec	name,description,arguments:list[ParameterSpec],completion:immediate/deferred,advances_clock:bool。
ParameterSpec	name,type,required,unit；适用时补 shape/enum/min/max/default，复合动作提供完整参数结构引用。
ClockSpec	{kind:dataset/simulation/wall,time_format:timestamp/offset,unit,origin}。timestamp 用带时区 RFC3339，统一交付 UTC，unit/origin 可为 null；offset 为数值，unit=s或step，origin 明示仿真/episode 起点。
ClockSnapshot	ClockSpec 加 now,visible_until,visible_boundary:exclusive/inclusive；分别是当前环境时间、已开放时间前沿及其边界。

Capabilities 字段	取值
start_modes	attach / reinitialize 的子集。
observe_modes	next / range / query 的子集。
progression	on_observe / on_action / external。
next_batching	time_window / transition：时间窗或原生转移。
history	none / session / source：无历史、会话缓存、源历史。
seed	boolean，声明能否按种子初始化，不承诺跨设备确定性。
delivery	inline / reference / both。
ordering	event_time / arrival：按事件时间或到达次序。

能力描述针对当前已解析配置。可推进仿真可以在 observe 时推进；原生 step 推进的后端保持 on_action；真实系统保持 external。纯观察任务使用 on_action 时，Runner 在初始状态后显式调用 default_command 并记录，不能把默认动作藏在 observe 中。

3.3 start：本次可读什么、运行到哪里

EnvironmentStart 字段	规则
run_id、mode	必需；mode 为 attach/reinitialize。
read_scope	必需 {start,end} 或 {duration}；同 ClockSpec 的时钟域。
visibility	causal / full_range；stream/interactive 用 causal，batch 的明确数据视图可用 full_range。
selection	{variables:list[string],entities:list[string]或null}；Runner 从 Binding 编译，entities=null 表示公开的全部适用实体。
seed、init	缺省 null、{}；seed 仅在 reinitialize 且能力支持时使用，init 格式由适配器公布。

Runner 根据任务预热和反馈需要编译 read_scope；execution.scope 控制产生新任务/动作的范围，feedback_until 只延长读取。on_action 后端没有独立可读反馈时，最后转移交付后结束，不为等待反馈追加动作。

SessionInfo 固定返回 `session_id,run_id,environment_id,source_revision,actual_mode,actual_scope:{start,end},clock,initial_cursor,seed_used,ownership:created/attached,public_context`。未知或不适用项为 null；duration 在 start 时换算实际范围，由 Runner 执行截止，Task 不另选起点。

回放定位固定数据版本与起点；仿真建立初态；Gym reset 后缓存初始 observation/info；Live attach 建立订阅或读取会话，不重置运行中的系统。历史预热需要相应 history 能力。

会话 start 后为 active；observe(next) 返回 end 后为 exhausted，可查已授权历史和 finish，不再 act；finish 后为 closed。同一实例不再次 start，新 Run 创建新会话。

3.4 observe：请求与返回

ObservationRequest 字段	规则
request_id、mode	request_id 由 Runner 分配；mode 为 next/range/query。
variables、entities	变量使用环境名，空 variables 表示仅取反馈/进展；entities=null 表示所选变量适用的全部已绑定实体。
after_cursor	next 使用会话最新游标；range/query 为 null。
advance_by	Duration；time_window 的 next 必需，transition 的 next 为 null。
range	range/query 必需 {start,end}，半开区间；next 为 null。
max_wait_ms	非负整数，缺省 1000；0 立即读取，是墙钟等待，不是仿真步长。
request_context	{case_id,turn_id} 或 null；用于关联当前任务的查询。

mode	交付内容	主时间/游标
next	下一时间窗或原生状态/转移的数据与反馈	按 progression 前进，成功交付或空窗进展更新游标。
range	batch 所需的授权数据片段或固定视图	只用于 full_range 会话；不推进、重置、回滚或改变主游标。
query	当前会话允许的既有片段	不扩展可见前沿、不消耗下一批缓存、不改变主游标；真实系统继续独立运行。

range/query 是一次只读请求，即使结果为空也返回 data，不用 end 表示查询完成。批量任务处理完计划片段即可 finish，不必再 next。查询范围或变量不被允许时明确报错，不返回伪装成合法的空数据。

ObservationBatch 字段	规则
session_id、request_id、batch_id	会话、请求与本次交付标识。
status、cursor	status=data/waiting/end；cursor 为最新主游标，range/query 返回原值。
clock	ClockSnapshot，反映本会话时间与可查询前沿。
coverage	time_window 的本次覆盖区间；range/query 为请求区间；不保证每点均有样本。
snapshot_time	transition 状态所处的 TimePoint；通常与 coverage 二选一，不适用项为 null。
data	TimeSeriesData、DataViewRef 或 null。
context、feedback	公共上下文 object（缺省 {}）；EnvironmentFeedback 列表（缺省 []）。
end	status=end 时必需 EndInfo，其他状态为 null；末批仍可带数据和反馈。

waiting 的 data/coverage/snapshot_time 均为 null、feedback=[]，cursor 和 visible_until 不变；wall 环境的 now 可以前进。data 可只含反馈或空窗进展。源耗尽后再次 next 返回相同结束原因，不重复末批内容。

EndInfo 为 `{reason,terminated,truncated,detail}`；reason 为 source_exhausted/read_scope_end/backend_terminated/backend_truncated，detail 可为 null。原生 terminated/truncated 均保留、可同时为 true；reason 说明主要原因，不把范围到期解释为任务成功。

3.5 观测格式与时间可见性

代码块

```
TimeSeriesData: {series: list[Series]}
Series:
  entity: string
  variable: string
  unit: string
  timestamps: list[TimePoint]
  values: list[scalar | tensor | null]
  valid_mask: list[bool]
  available_at: list[TimePoint] | null
```

timestamps/values/valid_mask 逐点等长，available_at 非 null 时也等长；单点类型与形状遵循 VariableSpec。mask=false 对应值为 null，mask 描述整个时点值。不同序列可不等间隔；缺失整行和显式无效点分别保留。适配器不自行做 Task 窗口、插值或聚合；非时序附件用公开 context 中的 ArtifactRef，结构化原生观测按公开字段映射，不丢弃未能映射的部分。

available_at 表示数据实际可被会话读取的时间。实时源使用后端时间或适配器收到时间；缺少该字段的历史数据可声明 event_time 假设并记录，不能据此声称恢复了实际传输延迟。

时间规则	统一含义
初始化前沿	causal 从 read_scope.start 开始，exclusive，首批尚未开放；full_range 直接开放授权终点，exclusive。
窗口前沿	time_window 的上界 exclusive；当前时刻 now 不等于数据都已可见。
状态前沿	transition 已交付状态时刻 inclusive；离散 step k 的前沿允许查询 [0,k+1) 内已开放步骤。
causal 读取	事件时间和 available_at 均须满足前沿，并满足 read_scope/selection；未来已知协变量由任务明确提供，不开放任意未来观测。
历史查询	实时源需支持按可用时间读取，或查询本会话已捕获缓存；每次实际返回仍记录，时间上界不等于数据库快照隔离。
终止边界	读取截止步 N 仍交付状态 s_N、奖励和终止信息，不能套半开窗口规则删掉最终状态。

对 time_window，设读取前沿为 p，目标 q=min(p+advance_by,read_scope.end)，本批 coverage=[p,q)。回放或可推进仿真完成此区间即推进，即使无样本也返回 data 与 coverage；外部源到达 q、完成该段读取后才交付，否则在 max_wait_ms 后返回 waiting。

采集窗口关闭不意味着迟到样本全部到齐。默认保留迟到数据，按到达顺序在后续批交付并保留 available_at，其事件时间可早于该批 coverage。Task 决定是否修订窗口，记录保存原交付顺序；丢弃策略须在 options 与摘要中说明数量。

transition 每次只交付一个状态/转移。on_action 的 start 缓存初态，成功 act 缓存下一转移；消费初态后未 act 再 next 是顺序错误。最多一个未交付转移，再次 act 前先 observe。原生 on_observe 后端才允许读取驱动转移；observe 取 Gym 缓存不再次 step。

cursor 是本会话的交付令牌，不是时间戳或跨 Run 恢复点，也不承诺跨后端恰好一次执行。首次 next 用 initial_cursor，其后用最近一次 next 的返回值；act 不改交付游标，waiting 不重发已接受动作。失效游标报 cursor_mismatch，不自动倒带。末次转移交付后不得再越界 act。

3.6 act：操作、回执与后续反馈

对象	字段与规则
EnvironmentCommand	operation_id 本 Run 唯一，由 Runner 分配；operation 使用已公布名称；target 为实体/对象，全局操作为 null；arguments 遵循 OperationSpec；output_id 关联原输出，默认推进动作为 null。
ActionReceipt	session_id,operation_id,output_id,state,received_at,completed_at,backend_ref,detail；时间使用环境时钟，后三项不适用为 null。output_id 从命令复制。
EnvironmentFeedback	feedback_id,operation_id,kind,occurred_at,available_at,payload；外生事件的 operation_id 可为空。kind=operation_status/reward/event。

回执 state	意义与后续
accepted	已接收、尚未完成；observe.feedback 收集状态更新。
completed	OperationSpec 定义的完成条件成立；继续观察系统结果，不等于业务目标达成。
rejected	明确未执行；Task 可形成纠正输入。
failed	操作失败；保留已知情况并结束本次控制运行。
unknown	无法确认是否执行；保留证据并结束本次控制运行，不自动重发。

operation_status 的 payload 为最新 state/detail，reward 为 name/value/unit，event 使用公开事件结构。operation_id 负责关联，不自动提供后端幂等。

Task/Binding 把任务操作转换为环境命令，适配器再转原生 API。即时回执由 TaskSession.on_action 接收并记录；后续 observe 只交付变化和新反馈。成功原生 step 的回执先登记，再读取转移后形成下一决策；rejected 无新转移时可继续纠正。末批 end 仍交给 Task 处理，但不再发新控制。

3.7 finish：结束会话及后端对应关系

EnvironmentSummary 返回 `session_id,actual_scope,last_cursor,final_clock,end,counts,pending_operations,artifact_refs,backend_disposition`。counts 为 batches/points/operations/late_points/dropped_points；pending_operations 是未完成操作 ID 列表；end 为 EndInfo 或 null，backend_disposition 为 closed/detached。

created 会话释放所建实例，attached 会话关闭连接或订阅并脱离，不关闭真实业务系统、不撤销已执行操作。finish 也用于启动失败的清理；重复调用返回缓存摘要，无 session 时相关字段可为空、计数为 0。上层停止原因由 RunResult.summary 说明。

后端	start / observe / act / finish 的映射
历史回放	定位数据版本与游标 / 按窗读取、授权范围查询 / 无反事实控制 / 关闭读取器。
可推进仿真	初始化 / 推进到目标时刻并采样 / 设置控制参数 / 关闭或脱离实例；若原生操作推进则改用 on_action。
Gym 环境	reset 缓存初态 / 交付缓存 / 每次 step 缓存一条转移 / close；一次 Run 对应一个 episode。
真实试验台或系统	attach / 按采集窗口读取 / 提交实际操作并追踪完成 / detached，系统继续运行。

Gym 的原生 info 只按明确公开字段映射到 context/feedback，其余可保留为受限调试证据，不整包透传为任务输入。

协议错误类别为 invalid_config、unsupported_capability、invalid_request、invalid_state、cursor_mismatch、backend_failure，携带 method/message 和适用的 request_id/operation_id。合法操作的 rejected/unknown 以回执表达。统一接口不保证各后端具有相同重置、延迟、动作集合或可重现性。

4. EnvironmentTS 内部：整理任务、调用方法、组织运行

4.1 Task 把观测变成任务和环境请求

代码块

```
class Task:
    def describe(self) -> TaskDescription: ...
    def bind(self, spec: TaskSpec, binding: BindingSpec,
             environment: EnvironmentDescription,
             execution: TaskExecutionContext) -> TaskSession: ...
class TaskSession:
    def next_observation(self) -> ObservationRequest: ...
    def consume(self, batch: ObservationBatch) -> TaskUpdate: ...
    def on_action(self, receipt: ActionReceipt) -> TaskUpdate: ...
    def interpret(self, task_input: TaskInput, output: CandidateOutput,
                  output_id: str) -> TaskEffect: ...
    def finish(self, reason: str) -> TaskSummary: ...
```

对象	字段或内容
TaskDescription	task_type,contract_version,input_schema_ref,output_schema_ref，以及支持的运行模板、请求、变量语义；TaskSpec.version 是实现版本，contract_version 是数据协议版本。
TaskExecutionContext	mode,scope,learning,feedback_until,data；data 仅暴露 read_view(ref: DataViewRef) -> TimeSeriesData，由 Runner 在 bind 时注入，不含凭证或私有评分依据。
TaskInput	case_id,turn_id,task_type,contract_version,payload,target,context；payload 是允许输入，target 明确结果对应的实体/变量/时间/对象，context 仅含公开上下文。
TaskUpdate	inputs:list[TaskInput],feedback:list[CandidateFeedback],phase:active/draining/complete；一次观测可暂不形成任务，也可产生多个输入。
TaskEffect	result:TaskResult或null,environment_request:EnvironmentCommand或ObservationRequest或null,case_complete:bool。
TaskResult	output_id,case_id,target 加任务协议规定的结果类型与内容，其结构由任务协议定义；目标键保持原预测/决策含义。
TaskSummary	已形成任务数、未完成任务和待到达结果，供运行收尾使用。

TaskSession 是逐 Run 的状态。consume 整理窗口及允许反馈，interpret 标准化结果并映射请求。方法自身的特征/学习留在其适配内部；统一任务输入也不强制使用某一种数组形状。

case_id 表示一个窗口、预测发出时点、诊断问题或控制决策。query 的 complete=false 会保持 case_id 并递增 turn_id；command 可按任务定义继续同一 case，也可完成本决策后使用新 case。基础模板先完成当前交互，再处理后续输入；原始输出分配 output_id 后先保存，标准化不使用后来信息改写原预测。

4.2 运行时调用对象的适配契约

这一小节定义 EnvironmentTS 怎样调用选定的实现。Candidate 可封装算法、模型、pipeline 或 Agent，保留原生计算和多轮状态。

代码块

```
class CandidateRegistry:
    def register(self, manifest: CandidateManifest,
                 factory: CandidateFactory) -> None: ...
    def resolve(self, adapter: str, version: str) -> CandidateManifest: ...
    def create(self, spec: CandidateSpec) -> Candidate: ...
class CandidateFactory:
    def create(self, spec: CandidateSpec) -> Candidate: ...
class Candidate:
    def prepare(self, context: CandidateContext,
                training: DataViewRef | None) -> PreparationResult: ...
    def infer(self, task_input: TaskInput) -> CandidateOutput: ...
    def update(self, feedback: CandidateFeedback) -> None: ...
```

```
13        def finish(self) -> list[ArtifactRef]: ...
```

CandidateManifest 字段	内容
adapter、version、candidate_kind	适配包身份；kind 为 algorithm/model/pipeline/agent，调用方法相同。
origin_ref	LibraryTS 条目、SkillTS 工作流、调用方实现或 Agent 服务的固定实现版本。
task_contracts	每项为 task_type/contract_version/input_schema_ref/output_schema_ref，与 TaskDescription 对应。
modes、learning、requests	支持的运行与学习方式；requests 每项为 kind/name/arguments_schema_ref，无请求则 []。
input_requirements	每个任务协议下的变量/单位、历史量、采样、缺失和实体要求。
params_schema_ref、artifact_schema_ref	参数与可加载制品格式；不需要制品格式时后者为 null。

扩展装载时登记，Runner 按 adapter/version resolve，匹配任务协议与输入条件，再每 Run create 一次。实现版本、适配版本、制品版本分别记录；方法和评分器分别通过固定版本工厂装配。

LibraryTS 接入时，`CandidateManifest.origin_ref` 关联固定的库条目/实现版本；`CandidateSpec.adapter/version` 选择已登记的调用适配，`artifact` 选择需要加载的模型或状态制品，`params` 指定本次参数。制品可来自库或此前训练，无需模型文件的函数使用 null。Registry 将匹配的 spec 交给已登记的 Factory，由其创建本 Run 的逻辑调用实例；prepare 完成制品加载或会话准备。适配层负责 TaskInput 与原生调用参数、原生返回与 CandidateOutput 的转换。一个库实现可供多个 Run 创建实例，一条流水线也可引用多个库组件；组件版本随实现或固定流水线制品记录。

方法或对象	契约
prepare	加载函数/模型/工具配置，建立本 Run 状态或 Agent 会话；需要时用指定训练材料拟合。
infer	接受本轮任务，调用算法或进行 Agent 规划/推理，输出结果或请求。
update	仅声明支持且 learning=online 时，用任务允许反馈学习；是可选方法。
finish	结束实例/会话，返回需保存的制品列表。
CandidateContext	run_id,candidate_spec,manifest,task,learning,training_targets,resource_policy_ref,io；task 含公开协议与本次绑定；io 仅提供第 2.3 节中本次获准的 resolve/read_view/save_artifact。
PreparationResult	{summary:object,artifact_refs:list[ArtifactRef]}。
CandidateOutput	{case_id,turn_id,result,request,complete}；result 为任务结果或 null，request 为一个 TaskRequest 或 null，complete 为本 case 是否完成。
TaskRequest	{kind:query/command,name,arguments}；使用任务名称和参数，经 Task/Binding 转换，由 Runner 执行。
CandidateFeedback	{feedback_id,output_ids,kind,payload,available_at}；按任务公开结构关联原输出，例如已观测预测目标或控制转移/奖励。

冻结 Agent 仍可积累本 Run 的上下文、记忆和规划状态，这不等于模型/策略训练。内部工具可由原平台处理既有输入；需要新环境数据或影响本实验系统的调用须返回 TaskRequest。多轮查询结果以同一 case 的下一 TaskInput 送回，不要求 Agent 改成单次分类器。跨 Run 继承模型或记忆须显式选择固定制品；BenchTS 私有真值不进入准备或更新。

4.3 Runner 的正常调用顺序

顺序	EnvironmentTS 内部动作
装配	解析计划和固定引用，创建 Environment/Task；describe、bind，解析并匹配调用对象的声明，再创建本次实例。
启动	records.begin_run；env.start，记录实际配置与会话；调用 prepare，记录训练来源和准备制品。
读取	task.next_observation → env.observe；先保存完整 observation，再 task.consume。
推理与反馈	按计划先交付允许的 update 反馈，再对 TaskInput 调用 infer；分配 output_id 并保存原始输出，然后 interpret。
请求	query → observe并记录 → consume → 同 case 下一轮；command → act并记录 → on_action，成功转移须先读取再继续控制。
收尾	先处理末批中范围内的有效任务和反馈，保存末态/末奖励；到期后不建立范围外决策。按计划读取后续反馈，调用各对象 finish，records.end_run，返回 RunResult。

batch 使用 range，完成所选片段后结束；stream 使用 next，窗口形成才调用；interactive 由 next 提供状态，query 支持同一任务多轮。末批 status=end 仍先 consume，并完成任务规则允许、决策时点在 execution.scope 内的非控制 TaskInput；预测按发出时点判断范围，不按未来目标时点裁剪。已终止环境不再 act。draining 阶段不产生新预测/动作；waiting 只继续读取。query、command 回执和末批触发的允许反馈按同一学习规则处理。

5. RunResult 与实验记录：交付结果及其来源

5.1 运行完成后交付什么

RunResult 字段	含义
run_id、purpose	本次运行标识；purpose 为 development/acceptance/regression，由调用路径确定。
completion	completed/scope_reached/failed；结束说明写入 summary，不把环境终止或范围到期直接解释成业务成功。
result_ref	本次 TaskResult 集合的固定入口。
artifact_refs	本次产生的 ArtifactRef 列表。
trace_ref	本次实验轨迹入口，解析到 EnvironmentTS.records；trace 表示轨迹，不是独立模块。
summary	输入/任务/结果数量、开销及结束说明；失败运行也保留已产生证据入口。

5.2 记录服务及对象

代码块

```python
class ExperimentRecords:
    def begin_run(self, meta: RunMetadata) -> RunRef: ...
    def append(self, run_id: str, events: list[TraceEvent]) -> None: ...
    def end_run(self, run_id: str, result: RunResult) -> None: ...
    def get_run(self, run_id: str) -> RunRecord: ...
    def read_events(self, run_id: str, query: EventQuery) -> EventPage: ...
    def attach_outcomes(self, run_id: str, outcomes: list[OutcomeRecord]) ->
None: ...
```

Runner 调用前三个方法组织实验证据；获准调用方读记录；结果采集或评价流程用 attach_outcomes 关联后到结果。Run 结束不禁止关联后续结果，也不因此自动调用方法学习或控制生产系统。

对象	内容
RunRef	{run_id,trace_ref}；begin_run 返回的标识和记录入口，后续 append/end_run 使用该 run_id。
RunMetadata	运行来源/用途、任务与场景版本、解析计划、CandidateManifest及实现版本、数据/训练/制品来源；导入生产片段时保留原运行标识、部署版本和范围。
TraceEvent	event_id,run_id,kind、记录时间、适用的环境时间；适用的 case_id,turn_id,output_id,operation_id,batch_id,request_id；完整内容或不可变制品引用。
EventQuery / EventPage	按种类、任务实例、时间选择；返回事件和下一页位置。
OutcomeRecord	outcome_id、关联的 output_id 或任务目标键、来源、可用时间、结果内容；更正时附 supersedes，保留旧记录。
RunRecord	RunMetadata、最终 RunResult（未结束时为空）、事件入口及后续结果关联。

kind 包含 observation、input、candidate_output、task_result、environment_operation、environment_feedback、candidate_update、artifact、run_end；可显式扩展方法内部阶段事件，由 Runner 关联保存。

observation 保存完整 ObservationBatch 或不可变快照，包含 data、clock、coverage/snapshot_time、feedback、EndInfo。末批即使未形成下一 TaskInput，也保留最终状态、最后奖励和 terminated/truncated。事件用 batch_id/request_id 关联读取；反馈以 operation_id 经操作记录回到 output_id。单列反馈事件与批次内相同反馈复用 feedback_id，不重复计奖。

Agent Platform/现场服务提供通用执行、权限、采集与存储；ExperimentRecords 补齐实验领域关联。只有明确选入实验的生产片段进入本次证据，不能把导入材料标成此次被测对象刚执行的结果。

实际流不能依赖可变查询路径复核，必要时保存观测快照。

6. BenchTS：使用固定结果和独立依据评价

配置或数据对象	字段或内容
EvaluationSpec	name（可选）、candidates,baselines,cases,scoring,comparison；候选引用解析为完整 CandidateSpec，案例固定场景和执行设置。
ScoringSpec	scorer,version,params，outcomes,metrics,aggregation,thresholds,missing_outcomes，以及结果截止时间；这些配置不自动交给被测对象。
ScoreSpec	已有 run_id 列表、固定 ScoringSpec、可选先前报告引用。
ResultView	原始输出、标准化结果、目标时间/对象以及运行/制品来源；只读。
OutcomeView	与目标对齐的真值、环境结果或其他评价依据及来源；只读，独立取得。
ScoringContext	案例分组、计量条件、任务/评分规则、预算和截止范围。
ScoreResult	指标值、单位、适用范围、计分数量、缺失数量与逐案例结果引用。
EvaluationReport	evaluation_id、候选和方案版本、案例/分组/基线结果、门槛判断、证据引用与未覆盖事项。

代码块

```
class Scorer:
    def score(self, results: ResultView, outcomes: OutcomeView,
              context: ScoringContext) -> ScoreResult: ...
```

BenchTS.evaluate 为固定候选/基线与案例组合生成 ExperimentSpec，经 EnvironmentTS 执行，再读取记录形成 ResultView。Scorer 将其与独立 OutcomeView 对齐评分，BenchTS 按 comparison 汇总。BenchTS.score 直接复用旧 Run，不要求方法生成“修正后预测”。

门槛判断为 pass/fail/insufficient_evidence；缺少真值或样本不足表达为证据不足，不用零替代。重评分生成新 evaluation_id，引用原 Run 和本次评分/结果版本；后到更正不覆盖原预测或旧报告。